У роботі деталізовано захист API backend системи MedRecordsManager. Основний результат - п’ять контролів із ДЗ5 перетворено на конкретні технічні рішення з порядком інтеграції, критеріями перевірки та оцінкою ресурсів. До MVP необхідні об’єктна авторизація, керування сесіями та захищений аудит.

1 Обраний компонент

API backend обробляє запити вебклієнта, мобільного застосунку й адмінпанелі, керує медичними записами та експортом PDF. Він критичний, оскільки саме тут перевіряється право користувача на конкретного пацієнта і дію; помилка може призвести до розголошення історій хвороб або підміни результатів аналізів.

Спадкоємність і технічні припущення

Зберігаю архітектуру ДЗ5: клієнти та адмінпанель → API → БД; окремо - IdP, черга експорту, приватне PDF-сховище, аудит і резервні копії. Пацієнт читає власні записи; лікар працює лише за чинним зв’язком із пацієнтом; адміністративна роль сама по собі не дає клінічного доступу.

Для конкретизації пропоную NestJS із TypeScript, PostgreSQL, Keycloak як IdP та Amazon S3 для приватних об’єктів. Це проєктний вибір для ДЗ7, а не твердження про вже впроваджений стек. Інфраструктурні сервіси розглядаються як залежності обраного API.

2 П’ять контролів безпеки

Контроль NIST Rev 5 [1]	**Застосування до API**
IA-5 - Authenticator Management	Керування автентифікаторами, токенами та їх відкликанням.
SC-23 - Session Authenticity	Перевірка справжності й чинності сесії на кожному запиті.
AC-3 - Access Enforcement	Право на конкретний запис, дію, клініку та експорт.
AU-12 - Audit Record Generation	Створення подій читання, зміни, відмови та експорту.
AU-9 - Protection of Audit Information	Захист журналу від зміни, видалення і зайвого читання.

Контролі взято з ДЗ5. Додаткові заходи нижче підтримують їх реалізацію; сам вибір інструментів не є доказом повної відповідності NIST.

3 Технічні рішення для ідентичності та доступу

IA-5 - Keycloak і керування життєвим циклом токенів

Рішення 1: Keycloak як готовий IdP з OIDC, централізованим блокуванням облікових записів і керуванням автентифікаторами. Реєстрацію та відновлення MFA виконувати після перевірки особи, без обходу через звичайне скидання пароля. Для персоналу налаштувати WebAuthn або пароль + TOTP. MFA перевіряє сервер IdP; frontend лише відображає процес. MFA персоналу також підтримує IA-2(1)/(2), а для пацієнтів застосовний контекст IA-8.

Рішення 2: access token живе не довше 15 хвилин; refresh token ротуються при використанні. Повторне використання вже витраченого refresh token має завершувати відповідну сесію та робити її недійсною для API. Це цільова поведінка інтеграції: її перевіряють для обраної версії IdP, а за потреби доповнюють обробником подій. Токени, паролі й OTP не потрапляють у логи. Так зменшується час використання викрадених облікових даних.

SC-23 - перевірка JWT і серверний реєстр сесій

Рішення 1: NestJS AuthGuard із бібліотекою jose перевіряє підпис JWT за довіреним JWKS, дозволений алгоритм, issuer, audience, exp і nbf. ID token не приймається замість access token. Кеш JWKS оновлюється при ротації ключів; невідомий ключ без успішного отримання довіреного JWKS означає відмову.

Рішення 2: PostgreSQL зберігає реєстр дозволених сесій за sid і sub. Після серверної перевірки входу створюється активна сесія; на кожному запиті API перевіряє її стан. Невідомий sid відхиляється, а не автоматично реєструється зі старого JWT. Logout спершу блокує sid у реєстрі, потім завершує сесію IdP; новий вхід отримує новий sid. Це реалізує також SC-23(1).

У MVP позитивний стан не кешується. Адміністративне відкликання записується через той самий сервіс; прямі події IdP синхронізуються і звіряються з затримкою не більше 60 с. При втраті актуальності понад цей поріг або недоступності реєстру захищені операції повертають 503. Лише строк дії JWT не забезпечує таке відкликання. MFA не зупиняє використання вже викраденого bearer token.

AC-3 - серверні політики і PostgreSQL RLS

Рішення 1: NestJS Guards і окремий PolicyService застосовують RBAC разом із перевіркою атрибутів та зв’язку лікар–пацієнт. Правило deny by default: роль дозволяє тип дії, але клініка, пацієнт і стан запису визначають доступ до об’єкта. OAuth 2.0 сам по собі не є об’єктною авторизацією.

Рішення 2: PostgreSQL Row-Level Security дублює ізоляцію клінік і пацієнтів. Backend працює без superuser, BYPASSRLS і прав власника таблиць; для відповідних таблиць увімкнено FORCE ROW LEVEL SECURITY. Довірений контекст задається через SET LOCAL усередині транзакції і не переноситься між запитами пулу. RLS - додатковий бар’єр; компрометація довіреного backend може обійти модель прикладного контексту.

4 Технічні рішення для аудиту

AU-12 - структуровані події і transactional outbox

Рішення 1: окремий AuditService формує події на межі бізнесоперацій, а Guard фіксує відмови в доступі. Поля: event_id, UTC timestamp, actor_id, session_id, clinic_id, action, resource_id, outcome, reason_code, correlation_id та ідентифікатори версій запису. Вхідні значення обмежуються за довжиною і серіалізуються як JSON, щоб запобігти ін'єкції в журнали.

Фіксуються успішні й невдалі входи, читання медзапису, зміни, виправлення, запит експорту, створення і завантаження PDF та зміни прав. Не записуються bearer tokens, cookie, паролі, повні діагнози чи тіла HTTP-запитів. Навіть ідентифікатори аудиту можуть бути персональними даними, тому журнал також має обмежений доступ.

Рішення 2: зміна медзапису та вставка події в outbox виконуються однією транзакцією PostgreSQL. Окремий worker передає події до незалежного сховища з повторними спробами та дедуплікацією за event_id. Для читання/експорту API підтверджує довговічний запис події перед видачею даних; якщо це неможливо, повертає 503. Збій віддаленого приймача допускається, поки локальний буфер справний і не переповнений.

AU-9 - окремі права і S3 Object Lock

Рішення 1: журнал приймає незалежний audit collector. Обліковий запис API може лише додавати події до outbox, без UPDATE/DELETE; worker має окремі мінімальні права. Читання журналу доступне аудитору, а керування інфраструктурою - окремому адміністратору. Події зміни цих прав теж надсилаються в аудит.

Рішення 2: collector архівує події в окремий приватний S3 bucket з Versioning і Object Lock. Для визначеного строку зберігання використовують compliance mode, який захищає відповідні версії об'єктів від зміни/видалення. Строк затверджується політикою клініки до ввімкнення; для навчального стенда можна перевірити 30 днів на синтетичних даних. Це проєктний параметр, не універсальна нормативна вимога.

API не має облікових даних для керування архівом. Ключі шифрування й права на їх видалення захищаються окремо: Object Lock не запобігає втраті читабельності через втрату ключа. Централізований перегляд можна додати через Loki/Grafana; пошуковий індекс не замінює захищений оригінал журналу.

Межі гарантій та реагування

Outbox гарантує зв'язок події зі штатною транзакцією, але не робить саму БД незмінною. До доставки в незалежне сховище лишається ризик видалення подій адміністратором БД. Для MVP worker надсилає їх безперервно; ціль затримки - до 60 с. Оповіщення про зростання черги, зникнення heartbeat, відмову запису чи зміну політик отримує відповідальний DevOps; він перевіряє доставку і відновлює її, не вимикаючи аудит.

5 Інтеграція рішень

Рівень	Розміщення і взаємодія
Frontend і мобільний клієнт	OIDC Authorization Code + PKCE; системний браузер для мобільного входу. UI не приймає остаточного рішення про права.
IdP	Keycloak перевіряє пароль/MFA, видає токени; події завершення сесій передаються до backend.
API backend	AuthGuard → перевірка сесії → PolicyService → валідація → транзакція БД → AuditService.
PostgreSQL	Медзаписи, зв'язки лікар–пацієнт, версії, сесії та outbox; приватна мережа, окремі службові ролі.
Черга і PDF worker	Завдання містить ID об'єкта й ініціатора, а не токен. Worker повторно перевіряє чинні права перед експортом.
Сховища і DevOps	Приватний PDF bucket, окремий audit bucket, TLS, секрети, резервування, моніторинг і контроль конфігурації.

Приклад зміни результату аналізу

1. Клієнт надсилає access token, ID результату, очікувану версію та нове значення. API перевіряє підпис, чинність сесії та право лікаря на цього пацієнта. Ідентичність і клініка беруться з довіреного серверного контексту, а не з полів запиту.

2. Backend перевіряє тип, одиниці, допустимі поля, зв'язок із замовленням і статус. У транзакції повторно перевіряє актуальні права та версію, щоб конкурентна зміна не перезаписала чужу роботу. Для завершеного результату створює нову версію з причиною виправлення, а не стирає попередню.

3. Подія зміни записується в outbox тією самою транзакцією. Після commit API повертає результат; worker доставляє аудит. Конфлікт версії повертає 409; недійсний токен - 401; заборонена дія - 403 або узгоджений 404 без розкриття існування чужого запису.

Експорт і залежності

PDF залишається приватним. Після повторної перевірки прав API передає файл користувачу; прямі публічні та presigned URL у цьому MVP не використовуються. Видалення зв'язку лікар–пацієнт блокує як новий експорт, так і завантаження раніше створеного PDF. На відповіді встановлюється Cache-Control: no-store.

Потрібні HTTPS, довірені сертифікати, синхронізація часу, JWKS, надійна БД сесій, менеджер секретів і службові ідентичності. Для вебклієнта доцільний BFF із HttpOnly/Secure/SameSite cookie та CSRF-захистом; мобільний refresh token зберігається в Keychain/Keystore. До запуску необхідні також резервні копії з перевіркою відновлення, обмеження запитів і валідація даних - п'ять обраних контролів не покривають увесь захист системи.

6 Ресурси і перевірка MVP

Оцінки нижче - орієнтовні людино-дні для навчального MVP з готовим CRUD, тестовим середовищем і доступною хмарною інфраструктурою. Вони не є оцінкою всього продукту чи виробничої сертифікації. Команда: backend-розробник, DevOps і QA; матрицю прав погоджує представник клініки.

Контроль	Ресурс і складність	Виклики та MVP
IA-5	3–5 днів Backend + DevOps	Середня. Відновлення MFA, UX входу, супровід IdP. Потрібно в MVP.
SC-23	3–5 днів Backend + QA	Середня. Синхронізація відкликання і залежність від БД. Потрібно в MVP.
AC-3	5–8 днів Backend + QA	Висока. Матриця прав, RLS, конкурентні зміни. Потрібно в MVP.
AU-12	3–5 днів Backend + QA	Середня. Повнота подій, outbox, дедуплікація. Потрібно в MVP.
AU-9	2–4 дні DevOps + QA	Середня. Розділення прав, retention, моніторинг доставки. Потрібно в MVP.

Разом 16–27 людино-днів, плюс приблизно 20–30% резерву на інтеграцію. Витрати залежать від хостингу IdP і БД, обсягу журналів, строку зберігання, запитів до сховища та підтримки. Open-source ліцензія не означає відсутність витрат на експлуатацію.

Критерії приймання

IA-5: прострочений токен відхиляється; повторне використання refresh token блокує сесію; токени і секрети відсутні у контрольній вибірці журналів.

SC-23: logout блокує наступний запит; адміністративне відкликання застосовується не пізніше 60 с; чужий audience, невідомий sid і неправильний підпис дають відмову. Недоступний реєстр не спричиняє перехід до перевірки лише підпису.

AC-3: пацієнт А не читає запис Б і не змінює навіть власний результат; лікар без чинного зв’язку отримує відмову; адміністратор не отримує клінічний доступ; export worker не створює PDF після відкликання прав. Тести виконуються напряму до API, в обхід UI.

AU-12: для дозволеної зміни існують нова версія та відповідна подія; rollback не залишає події успішної зміни. Недоступний collector не губить outbox; повторна доставка не створює дублікати в представленнях журналу.

AU-9: API не може змінити/видалити журнал або прочитати архів; видалення захищеної версії об’єкта блокується. Контрольна подія доходить до архіву, а затримка понад поріг викликає сповіщення.

Допустимі компроміси

Можна відкласти складний SIEM, власний policy engine та кластер Vault, використавши прості серверні політики і керований менеджер секретів. Не можна відкладати авторизацію кожного об’єкта, відкликання сесій, приватність PDF чи базовий захищений аудит. Відмови залежностей погіршують доступність: для клінічної експлуатації потрібен окремо погоджений аварійний процес доступу, а не автоматичний обхід перевірок.

7 Додатковий сценарій зовнішня лабораторія

Клініка передає лабораторний модуль зовнішньому вендору. Це нова довірча межа: стороння система надсилає результати через окремий інтеграційний API. Обрані вище контролі поширюються на новий канал, але сервісна ідентичність не підміняє лікаря.

Новий ризик	**Необхідна зміна**
Компрометація облікових даних вендора	Окремий OAuth client credentials клієнт для кожного вендора; короткий токен з audience лабораторного API і вузьким scope results:submit. Ключі ротуються; доступ можна негайно заблокувати.
Доступ до чужих пацієнтів або замовлень	Перевірка vendor_id, clinic_id, order_id і дозволеного дослідження на сервері. Надання мінімуму даних; без доступу до всіх історій хвороб і масового експорту.
Повторне надсилання або підміна результату	mTLS для каналу; idempotency key з унікальним обмеженням у БД; перевірка схеми, одиниць, статусу замовлення. Конфліктний результат потрапляє на ручну перевірку.
Помилка вендора або простій інтеграції	Черга, обмежені повторні спроби, dead-letter queue, rate limits і сповіщення. Критичні результати/виправлення підтверджує уповноважений працівник.
Витік і недостатня простежуваність	Audit event містить vendor_id, order_id, correlation_id і результат перевірки. Узгоджуються строки зберігання, видалення, повідомлення про інциденти та правила роботи субпідрядників.

Для MVP інтеграції обов’язкові окрема ідентичність, вузькі права, серверна валідація, захист від повторної обробки й аудит. До реальних даних проводиться тест на синтетичних записах. Навіть справжній mTLS-клієнт може надіслати помилковий результат, тому криптографічна справжність не замінює клінічну перевірку.

8 Висновок

Для API backend обов’язковими є контроль життєвого циклу автентифікаторів, перевірка активної сесії, авторизація конкретного запису та створення захищеного аудиту. Компроміс можливий у складності інструментів моніторингу й автоматизації, але не в правилах доступу чи приватності медичних файлів. Залишаються ризики використання викраденої активної сесії до її блокування, помилок і зловживань уповноважених працівників, компрометації backend, втрати ще не доставленого аудиту та копіювання даних законним отримувачем. Їх зменшують перегляд прав, контроль аномалій, незалежне погодження критичних змін і підготовлений процес реагування.